EcoLoop: Verbatim Spoken Recording Script

Target: 15 to 18 Minutes
Swami Sant Dass Public School

Complete read-aloud script for video / audio presentation recording.

> **Delivery Guidelines:** Speak at a calm, deliberate, conversational pace (around 125 words per minute). Do not shout or sound like a motivational speaker. Speak with deadpan confidence, like an engineer walking through a system they designed and built. Watch for `[ADVANCE SLIDE]` cues to time your slide clicks with your speech.

`SLIDE 01 • TITLE`

EcoLoop: Circular material systems and zero-cost digital architecture

"Hello everyone, and welcome. My name is Harman, and today I'm presenting **EcoLoop**, an initiative developed at Swami Sant Dass Public School in Jalandhar. **(short pause)**"

"Most student environmental projects stop at collecting garbage or making posters. We wanted to treat sustainability as a genuine **systems engineering problem**. That means solving two specific challenges: first, taking non-recyclable physical waste and transforming it into real, load-bearing utility materials. And second, building a modern digital platform to catalog, verify, and distribute those materials with exactly **zero dollars** in operating costs."

"Let's look at the physical material reality first." `[ADVANCE SLIDE]`

`SLIDE 02 • THE PROBLEM`

The Linear Packaging Problem

"When people talk about plastic waste, they usually talk about beverage bottles. But the biggest unaddressed crisis in modern packaging is **Multi-Layer Packaging**, or MLP. These are your chips packets, chocolate wrappers, and snack pouches."

"They fuse plastic films with microscopic layers of aluminum foil. Because those materials are chemically bonded, municipal recycling plants cannot separate them cost-effectively. As a result, **almost 100% of multi-layer packaging goes straight to landfills** or gets incinerated."

"At the same time, schools and communities generate tons of this clean packaging every week, but lack any logistics connecting them to upcycling facilities. With EcoLoop, our response is direct mechanical upcycling: we heat-compress that clean packaging into dense structural boards that directly replace commercial plywood." `[ADVANCE SLIDE]`

`SLIDE 03 • PHYSICAL PRODUCTS`

The Physical Loop: Verified Materials

"Here you can see the six circular products we produce and manage in our system."

"On the right is an actual sample of our **MLP Sheet**. That entire rigid board is made out of compressed snack packaging. It is completely waterproof, 100% termite-proof, and can be sawed, drilled, and nailed using standard woodworking tools."

"Alongside that, we produce **Eco-Bricks** packed with clean flexible films, **Bottle Planters** for nursery gardening, **Uniform Bags** stitched from discarded school uniform fabrics, **Seed Paper** embedded with native seeds, and citrus-fermented **Bioenzymes** for natural cleaning. Every single item here diverts a specific municipal waste stream into a practical physical product." `[ADVANCE SLIDE]`

`SLIDE 04 • TRANSITION 01`

Reality Check: How It Started vs How It Is Going

"Now, a quick reality check on how this project actually evolved. **(slight deadpan smile)**"

"When we first proposed this at school, everyone assumed we were just going to put three colored dustbins in the hallway and pick up trash after lunch."

"Within two weeks, we realized that tracking inventory, publishing material specs, and coordinating with community partners meant we needed an entire distributed cloud software system. So we ended up spending our evenings writing code instead of just collecting wrappers. Let's look at that digital side." `[ADVANCE SLIDE]`

`SLIDE 05 • DIGITAL LAYER`

Why Physical Projects Need Software

"Why does a physical recycling project need custom software in the first place?"

"Because without rigorous engineering specifications, recycled materials remain a school craft. Nobody takes them seriously for real construction or institutional purchasing. On our web application, every material listing includes **dimensions, density ratings, thermal properties, and manufacturing lead times**."

"Community partners and schools can explore the catalog, inspect real specs, and submit direct sample orders. We engineered the frontend using **React 18, TypeScript, and Vite** to ensure sub-50-millisecond page loads and strict type safety." `[ADVANCE SLIDE]`

`SLIDE 06 • SERVERLESS BACKEND`

The Zero-Cost Serverless Backend

"Our primary engineering constraint for the digital platform was straightforward: **we had a zero-dollar budget**."

"High school students do not have corporate credit cards to fund $50 monthly cloud hosting, container services, or database clusters. So we designed a lean serverless pipeline:"

"Step 1: The user submits an inquiry form on the React client, which validates all inputs locally. Step 2: The payload dispatches to a stateless Google Apps Script webhook running securely in the cloud. Step 3: That webhook appends structured records directly into a Google Sheet, which acts as our relational datastore and triggers instant email alerts to our team. **(pause)** Total ongoing infrastructure cost: **exactly zero dollars per month**, with 99.9% uptime and zero server maintenance." `[ADVANCE SLIDE]`

`SLIDE 07 • TRANSITION 02`

Reality Check: The Cloud Bill Dilemma

"Which brings us to reality check number two. **(deadpan delivery)**"

"You see enterprise software developers on Twitter debating for six straight months about microservices architecture, Docker containers, and Kubernetes pods, racking up a $450 monthly AWS bill."

"Meanwhile, two high schoolers run their entire transactional order pipeline on a free Google Sheet, and it has never once crashed." `[ADVANCE SLIDE]`

`SLIDE 08 • INQUIRY ENGINE`

Interactive Inquiry Engine

"Here you can see the actual inquiry engine in action on the right."

"When an institution or student clicks 'Enquire' on any product, the interface preserves state. It automatically pre-populates the product ID, specifications, and suggested minimum quantities directly into the modal."

"Client-side validation catches formatting errors instantly before any network call is made. Once submitted, the user sees an immediate confirmation modal with estimated delivery lead times, while the transaction is instantly synchronized to our central spreadsheet." `[ADVANCE SLIDE]`

`SLIDE 09 • PARADIGM SHIFT`

The Paradigm Shift: Developer as Architect

"Now let's talk about how this software was actually created, because how we write code has undergone a fundamental transformation."

"The traditional method of coding meant sitting down, manually typing boilerplate line by line, spending three hours on Stack Overflow trying to fix broken package imports, and constantly fighting syntax errors."

"The modern paradigm is **Developer as Architect**. You don't spend your time typing boilerplate. You define the system constraints, specify the data contracts, and direct autonomous AI agents to write the code across the codebase. Your human focus shifts entirely to architecture, design rules, and verification." `[ADVANCE SLIDE]`

`SLIDE 10 • TRANSITION 03`

Reality Check: Learning to Code in 2026

"Reality check number three."

"The traditional advice given to students was: 'Read 400 pages of official framework documentation before you build your first app.'"

"Sitting at a desk at 3:00 AM memorizing API manuals is officially legacy behavior. Today, if you know how to write clear specifications and system constraints, you can build production-ready applications in days instead of months." `[ADVANCE SLIDE]`

`SLIDE 11 • AGENTIC LOOP`

How the Autonomous Coding Loop Works

"This is not copy-pasting code snippets from a chatbot. It is a **closed-feedback engineering cycle** with four stages:"

"Stage 1: Define the Goal. You write a clear specification document outlining system rules, allowed libraries, and component contracts."

"Stage 2: Autonomous Edit. The agent reads your file tree, plans architectural changes, and edits multiple files across the repository."

"Stage 3: Automated Check. Automated tools compile the project. If there is a build error or broken import, the agent reads the terminal output and repairs the bug itself."

"Stage 4: Safe History. Every working state is saved as an atomic Git checkpoint, meaning you can experiment fearlessly because you can rollback anytime." `[ADVANCE SLIDE]`

`SLIDE 12 • OPENCODE`

Introducing OpenCode

"To implement this workflow, we specifically recommend an open-source tool called **OpenCode**, created by Anomaly."

"It is 100% open source on GitHub at `anomalyco/opencode`. Unlike web chatbots, OpenCode runs directly in your terminal. It features dual execution modes:"

"'Plan Mode' explores your codebase, reviews architecture, and plans changes without touching a single file. 'Build Mode' executes those changes, creates components, and runs tests. You can install it on any Mac, Linux, or Windows machine in 60 seconds using `npm install -g @opencode/cli`." `[ADVANCE SLIDE]`

`SLIDE 13 • FREE MODELS`

Free AI Models: Zero Cost for Students

"The best part for students is that you do not need expensive subscriptions to use autonomous coding tools."

"You have two completely free options: First, **Free Cloud APIs**. Google AI Studio provides generous free tiers for Gemini 1.5 Flash and Gemini 2.0. Groq Cloud offers ultra-fast token generation on open models like Llama 3.3. And GitHub Models gives students direct access to models like GPT-4o inside VS Code."

"Second, you can run **100% Offline with Ollama**. You download Ollama directly to your laptop, pull an open coding model like `qwen2.5-coder:7b`, and code with an autonomous agent completely offline with zero latency and zero internet connection required." `[ADVANCE SLIDE]`

`SLIDE 14 • TRANSITION 04`

Reality Check: The Compiler Never Lies

"And finally, our fourth and most important reality check. **(slight chuckle / deadpan)**"

"The AI model will smile and tell you: 'I have carefully resolved all bugs, everything is fully optimized!' And then you run your TypeScript compiler and find 47 errors across 3 files."

"The rule of thumb for modern engineering is simple: never trust what an AI claims it wrote. Always let your compiler, type checker, and automated test suite be the final judges of truth."
`[ADVANCE SLIDE]`

`SLIDE 15 • CONCLUSION & QR`

Thank You & Open Discussion

"To wrap up: EcoLoop demonstrates that students can take on complex physical material problems while building robust, production-grade digital software for zero dollars using modern open-source tools."

"If you scan the QR code on the screen, it takes you directly to our GitHub repository at `github.com/Harman209/ecoloop`. There you will find the complete open-source codebase, the live web catalog, and a one-page setup guide with exact terminal commands to set up OpenCode and free AI models on your own machine tonight."

"Thank you so much for your time, and I would love to answer any questions or hear about the environmental and software projects you are building." **(conclude recording)**